

Lab01-AlgorithmAnalysis

CS2312-Algorithm and complexity, Xiaofeng Gao, Spring 2026.

* Contact TA Steven Chen for any questions. Include both your .pdf and .tex files in the uploaded .rar or .zip file.
* Name: Hengtao Wu Student ID: 524031910038 Email: Eternity_w@sjtu.edu.cn

1. (Minimal Counterexample Principle) Prove that if one is given an unlimited supply of 6 cents, 10 cents, and 15 cents, one can make any amount of change larger than 29 cents.

Proof. Assume for contradiction that there exists an integer $n > 29$ that cannot be expressed in the form

$$n = 6i + 10j + 15k,$$

for nonnegative integers i, j, k . Let n be the smallest such integer.

First, observe that the following values can all be expressed:

$$30 = 6 \cdot 5, \quad 31 = 6 + 10 + 15, \quad 32 = 6 \cdot 2 + 10 \cdot 2, \quad 33 = 6 \cdot 3 + 15, \quad 34 = 6 \cdot 4 + 10, \quad 35 = 10 \cdot 2 + 15.$$

Hence, every integer from 30 to 35 is representable, so the smallest counterexample must satisfy $n \geq 36$. Therefore, $n - 6 \geq 30 > 29$. By the minimality of n , the integer $n - 6$ is not a counterexample and can be written as

$$n - 6 = 6i + 10j + 15k,$$

for some nonnegative integers i, j, k .

Then

$$n = 6(i + 1) + 10j + 15k,$$

which shows that n is also representable. This contradicts the assumption.

Therefore, no such counterexample exists, and every integer $n > 29$ can be written as a combination of 6, 10, and 15. $\square$

2. (Complexity Class) Given the following functions:

$$f_1(n) = \lg n, \quad f_2(n) = 2^n, \quad f_3(n) = 1, \quad f_4(n) = \log^2 n$$

$$f_5(n) = n, \quad f_6(n) = n!, \quad f_7(n) = n^n, \quad f_8(n) = 2^{\sqrt{\log n}}$$

- (a) Assume $\log n = \log_2 n$ and $\lg n = \log_{10} n$. Rank the functions $f(n)$ by growth rate, with " $\succ$ " and " $\equiv$ ". e.g. $n^2 \succ n \equiv 4n$.

Solution.

$$f_7(n) \succ f_6(n) \succ f_2(n) \succ f_5(n) \succ f_8(n) \succ f_4(n) \succ f_1(n) \succ f_3(n)$$

Brief Justification:

- $f_3(n) = 1$: Constant time, the slowest.
- $f_1(n) = \lg n$: Logarithmic time, $\lg n = \Theta(\log n)$.
- $f_4(n) = \log^2 n$: Power of logarithm dominates standard logarithm.

- $f_8(n) = 2^{\sqrt{\log n}}$: Let $k = \log n$. We compare k^2 and $2^{\sqrt{k}}$. Exponential of $\sqrt{k}$ grows faster than polynomial of k , so $2^{\sqrt{\log n}} \succ \log^2 n$.
- $f_5(n) = n = 2^{\log n}$: Since $\log n \succ \sqrt{\log n}$, $2^{\log n} \succ 2^{\sqrt{\log n}}$. Thus $n \succ 2^{\sqrt{\log n}}$.
- $f_2(n) = 2^n$: Exponential function strictly dominates any polynomial function.
- $f_6(n) = n!$: $n! \sim \sqrt{2\pi n}(n/e)^n$, which grows strictly faster than 2^n .
- $f_7(n) = n^n$: $n^n = n \cdot n \cdots n > n \cdot (n-1) \cdots 1 = n!$.

□

3. (Time Complexity of Attention)

Attention is a fundamental component of the Transformer architecture. The standard *scaled dot-product attention* is defined as

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (1)$$

where Q , K , and V denote the *query*, *key*, and *value* matrices, with d_k representing the dimensionality of both Q and K , and d_v representing the dimensionality of V .

Since the representational capacity of a single attention head is limited, modern Transformer models employ *Multi-Head Attention*. This mechanism applies multiple attention operations in parallel and concatenates their outputs:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O \quad (2)$$

where each attention head is defined as

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V), \quad (3)$$

and W_i^Q, W_i^K, W_i^V are learned parameters for the i -th head, while W^O is the learned output projection matrix applied to the concatenated multi-head outputs. These W matrices are not fixed inputs.

Assume the following for the input sequence $X \in \mathbb{R}^{n \times d_m}$:

- The input sequence length is n .
 - The model dimension is d_m .
 - The number of attention heads is h .
 - Each head has dimensionality $d = d_m/h$.
 - The *query* and *key* dimensionalities are equal: $d_k = d_v = d$.
- (a) Write the pseudo-code for the **Multi-Head Attention** and **Scaled Dot-Product Attention** functions. Assume the input sequence X has already been linearly projected into Q , K , and V . Clearly specify the input and output variable domains (i.e., dimensions) in your algorithm. For **Multi-Head Attention**, you can use *Attention()* as a black box function that computes scaled dot-product attention.
 - (b) Analyze the **time complexity** of Scaled Dot-Product Attention in terms of n and d .
(Detailed explanation required)
(HINT: $d_k = d_v = d$)

- (c) Analyze the **time complexity** of Multi-Head Attention in terms of n and d_m .
 (Detailed explanation required)
 (HINT: In practice, $n \gg d_m$. Consider relationship between d and d_m)

Solution. (a) The pseudo-code for **Scaled Dot-Product Attention** and **Multi-Head Attention** is presented below.

Algorithm 1 Scaled Dot-Product Attention

Input : Query $Q \in \mathbb{R}^{n \times d}$, Key $K \in \mathbb{R}^{n \times d}$, Value $V \in \mathbb{R}^{n \times d}$

Output: Context matrix $O \in \mathbb{R}^{n \times d}$

```

1  $Scores \leftarrow \frac{QK^T}{\sqrt{d}};$  // Matrix mult:  $(n \times d) \times (d \times n) \rightarrow n \times n$ 
2  $Weights \leftarrow \text{softmax}(Scores, axis=-1);$  // Domain:  $n \times n$ 
3  $O \leftarrow Weights \cdot V;$  // Matrix mult:  $(n \times n) \times (n \times d) \rightarrow n \times d$ 
4 return  $O$ 
```

Algorithm 2 Multi-Head Attention

Input : Sequence $X \in \mathbb{R}^{n \times d_m}$, heads h , $d = d_m/h$

Weight matrices $W_i^Q, W_i^K, W_i^V \in \mathbb{R}^{d_m \times d}$ for $i \in \{1, \dots, h\}$

Output weight matrix $W^O \in \mathbb{R}^{d_m \times d_m}$

Output: Final representation $Final_Output \in \mathbb{R}^{n \times d_m}$

```

1 for  $i \leftarrow 1$  to  $h$  do
2    $Q_i \leftarrow XW_i^Q;$  // Matrix mult:  $(n \times d_m) \times (d_m \times d) \rightarrow n \times d$ 
3    $K_i \leftarrow XW_i^K;$  // Matrix mult:  $(n \times d_m) \times (d_m \times d) \rightarrow n \times d$ 
4    $V_i \leftarrow XW_i^V;$  // Matrix mult:  $(n \times d_m) \times (d_m \times d) \rightarrow n \times d$ 
5    $head_i \leftarrow \text{Attention}(Q_i, K_i, V_i);$  // Domain:  $n \times d$ 
6
7  $ConcatOutput \leftarrow \text{Concat}(head_1, \dots, head_h);$  // Domain:  $n \times (h \cdot d) \equiv n \times d_m$ 
8  $FinalOutput \leftarrow ConcatOutput \cdot W^O;$  // Matrix mult:  $(n \times d_m) \times (d_m \times d_m) \rightarrow n \times d_m$ 
9 return  $FinalOutput$ 
```

- (b) The time complexity of **Scaled Dot-Product Attention** is $\mathcal{O}(n^2d)$. The computation consists of three main steps:

- **Step 1 (QK^T):** Multiplying the query matrix $Q \in \mathbb{R}^{n \times d}$ and the transposed key matrix $K^T \in \mathbb{R}^{d \times n}$ takes $\mathcal{O}(n \cdot d \cdot n) = \mathcal{O}(n^2d)$. The scaling by $\sqrt{d}$ takes $\mathcal{O}(n^2)$.
- **Step 2 (softmax):** Applying the softmax function to the $n \times n$ score matrix takes $\mathcal{O}(n^2)$.
- **Step 3 ($Weights \cdot V$):** Multiplying the $n \times n$ weight matrix with the value matrix $V \in \mathbb{R}^{n \times d}$ takes $\mathcal{O}(n \cdot n \cdot d) = \mathcal{O}(n^2d)$.

Summing these up, the dominant term is clearly the matrix multiplication, yielding a total time complexity of $\mathcal{O}(n^2d)$.

- (c) The time complexity of **Multi-Head Attention** is $\mathcal{O}(n^2d_m)$. The computation consists of four main steps:

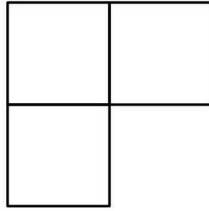
- **Step 1 (XW_i):** During the loop, multiplying the input matrix $X \in \mathbb{R}^{n \times d_m}$ with the projection matrices $W_i^{Q/K/V}$ takes $\mathcal{O}(n \cdot d_m \cdot d)$. Since the loop runs h times and $h \cdot d = d_m$, this step takes a total of $\mathcal{O}(n \cdot d_m^2)$.

- **Step 2 (Attention):** From Question 3b, we know that the *Attention* operation in each iteration takes $\mathcal{O}(n^2d)$. Hence, loop for all h heads takes $\mathcal{O}(n^2 \cdot d \cdot h) = \mathcal{O}(n^2d_m)$.
- **Step 3 (Concat):** Because this step only needs to concatenate the h matrices, it takes $\mathcal{O}(n \cdot d \cdot h) = \mathcal{O}(nd_m)$.
- **Step 4 (Final):** Similar to the matrix multiplications discussed above, multiplying the concatenated matrix with the output matrix takes $\mathcal{O}(n \cdot d_m \cdot d_m) = \mathcal{O}(nd_m^2)$.

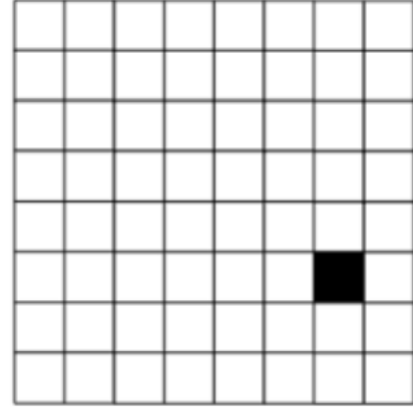
In summary, since $n \gg d_m$ in practice, the term $\mathcal{O}(n^2d_m)$ grows much faster than $\mathcal{O}(nd_m^2)$. This indicates that Step 2 is the dominant, bringing the overall time complexity of Multi-Head Attention to $\mathcal{O}(n^2d_m)$.

□

4. **(Divide-and-Conquer Analysis)** A tromino is an L -shaped tile formed by three connected 1×1 squares. Given an $n \times n$ (where $n = 2^k$, $k \geq 1$) board with a missing square cell, your task is to fill the remaining cells with trominoes. Trominoes can be oriented in any arbitrary way, but they should cover all the cells of the board except the missing one exactly and with no overlaps.



(a) Tromino



(b) Board

- (a) Design a divide-and-conquer method for the tromino tiling problem. Explain your algorithm. Do not provide pseudo-code.

(HINT: Place the first tromino at the center of board)

- (b) Express the cost of solving the problem as a recurrence relation and use the Master Theorem to find the running time.

Solution. (a) **Divide-and-Conquer Algorithm:**

- **Base Case:** First, we establish the base case. When $k = 1$ (i.e., $n = 2$), the board has exactly one missing cell, and the remaining three cells naturally form an L-shape.
- **Divide and Place:** For a general $n \times n$ board, we logically divide it into four $(n/2) \times (n/2)$ quadrants. We place the first tromino at the exact center of the board. We then adjust its orientation so that its three squares fall into the three quadrants that do *not* contain the originally missing cell.
- **Conquer (Recursive Step):** If we treat the originally missing cell and the three cells covered by the center tromino all as "occupied" cells, we can observe that each of the four quadrants now has exactly one occupied cell and a size of $n' = n/2$. We

can then recursively apply this exact algorithm to fill the remaining cells in each of the four quadrants until we reach the base case.

- (b) Since the problem is divided into four subproblems at each step, with the problem size reduced to $n/2$, and each merge operation involves placing only a single tromino (i.e., constant time $\mathcal{O}(1)$), we can formulate the following recurrence relation:

$$T(n) = 4T\left(\frac{n}{2}\right) + \mathcal{O}(1).$$

According to the Master Theorem, where $f(n) = \mathcal{O}(n^d)$ with $d = 0$, we have $d = 0 < \log_b a = \log_2 4 = 2$. Therefore, we get $T(n) = \mathcal{O}(n^{\log_b a}) = \mathcal{O}(n^2)$.

□